CSE563: Unit II Study Notes

Vectors and Matrices

School of Computer Science & Engineering

Session 2025–26

Contents

1 Unit II: Vectors and Matrices **2**
1.1 Vectors and Vector Operations . . . 2
1.2 Matrices and Matrix Operations . . . 3
1.2.1 Eigenvalues and Eigenvectors . . . 4
1.3 Principal Component Analysis (PCA) . . . 5
1.4 Linear Discriminant Analysis (LDA) . . . 6
1.5 Image Transformation . . . 7
1.6 Image Interpolation . . . 8
1.7 Unit II Quick-Revision Sheet . . . 9

1 Unit II: Vectors and Matrices

Course Outcome CO2: demonstrate the use of vectors and matrices in image transformations.

A digital image is a matrix. A data set is a matrix (rows = samples, columns = features). A rotation is a matrix. Linear algebra is therefore the language of computer vision and machine learning, and this unit builds from vector operations up to two matrix-based algorithms (PCA and LDA) and two image operations (geometric transformation and interpolation).

1.1 Vectors and Vector Operations

Definition 1.1. A **vector** $\mathbf{v} \in \mathbb{R}^n$ is an ordered list of n real numbers, $\mathbf{v} = (v_1, \ldots, v_n)^T$, written as a column. Geometrically it is an arrow with a *magnitude* and a *direction*; algebraically it is a point in n-dimensional space. A **scalar** is a single number.

Formula & Mathematical Relation: Vector Operations

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and scalar c:

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_n + v_n) \quad \text{(component-wise; parallelogram law)} \tag{1}$$

$$c\,\mathbf{v} = (cv_1, \ldots, cv_n) \quad \text{(stretches, } c < 0 \text{ reverses)} \tag{2}$$

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{n} u_i v_i = \mathbf{u}^T \mathbf{v} \quad \text{(dot / inner product, a scalar)} \tag{3}$$

$$\|\mathbf{v}\|_2 = \sqrt{\mathbf{v} \cdot \mathbf{v}} = \sqrt{\sum v_i^2} \quad \text{(Euclidean norm / length)} \tag{4}$$

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|} \quad \text{(angle between vectors)} \tag{5}$$

$$\text{proj}_{\mathbf{u}}\,\mathbf{v} = \frac{\mathbf{v} \cdot \mathbf{u}}{\|\mathbf{u}\|^2}\,\mathbf{u} \quad \text{(projection of } \mathbf{v} \text{ onto } \mathbf{u}) \tag{6}$$

Other norms: $\|\mathbf{v}\|_1 = \sum |v_i|$ (Manhattan), $\|\mathbf{v}\|_\infty = \max |v_i|$. A **unit vector** has norm 1: $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$.

What the Dot Product Tells You

- $\mathbf{u} \cdot \mathbf{v} = 0 \iff$ the vectors are **orthogonal** (perpendicular).
- $\mathbf{u} \cdot \mathbf{v} > 0$: angle less than 90° ("pointing the same way"); < 0: more than 90°.
- $\mathbf{u} \cdot \hat{\mathbf{v}}$ is the length of the shadow of $\mathbf{u}$ on the direction $\hat{\mathbf{v}}$. *This single fact is PCA*: projecting data onto a principal direction is a dot product.
- In ML, **cosine similarity** $= \cos\theta$ compares documents/embeddings independently of their length.

Worked Example: Vector Arithmetic

$\mathbf{u} = (3, 4)$, $\mathbf{v} = (1, -2)$. $\mathbf{u} + \mathbf{v} = (4, 2)$; $2\mathbf{u} - \mathbf{v} = (5, 10)$; $\mathbf{u} \cdot \mathbf{v} = 3 - 8 = -5$; $\|\mathbf{u}\| = 5$, $\|\mathbf{v}\| = \sqrt{5}$; $\cos\theta = \dfrac{-5}{5\sqrt{5}} = -\dfrac{1}{\sqrt{5}} \approx -0.447$, so $\theta \approx 116.6°$. Projection of $\mathbf{v}$ onto $\mathbf{u}$: $\dfrac{-5}{25}(3, 4) = (-0.6, -0.8)$. Unit vector along $\mathbf{u}$: $(0.6, 0.8)$.

Cross Product (3-D only)

$\mathbf{u} \times \mathbf{v} = (u_2v_3 - u_3v_2,\ u_3v_1 - u_1v_3,\ u_1v_2 - u_2v_1)$ is a vector *perpendicular* to both, with $\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\|\|\mathbf{v}\| \sin\theta$ (area of the parallelogram). Used in graphics for surface normals and in

physics for torque. Example: $(1,0,0) \times (0,1,0) = (0,0,1)$.

Linear Combination, Span, Independence, Basis

- A **linear combination** is $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$. The set of all such combinations is the **span**.
- Vectors are **linearly independent** if $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ only when all $c_i = 0$ (no vector is a combination of the others). $(1,2)$ and $(2,4)$ are dependent; $(1,0)$ and $(0,1)$ are independent.
- A **basis** of $\mathbb{R}^n$ is a set of n independent vectors; every vector is a unique combination of them. Standard basis: $\mathbf{e}_1, \ldots, \mathbf{e}_n$. An **orthonormal** basis has mutually perpendicular unit vectors—PCA constructs one.
- **Dimension** of a space = number of vectors in any basis.

1.2 Matrices and Matrix Operations

Definition 1.2. An $m \times n$ **matrix** $A = [a_{ij}]$ is a rectangular array with m rows and n columns. Special types: *square* ($m = n$); *identity* I (ones on the diagonal); *diagonal*; *zero*; *symmetric* ($A = A^T$); *upper/lower triangular*; *orthogonal* ($A^TA = I$, i.e. $A^{-1} = A^T$, columns are an orthonormal basis).

Formula & Mathematical Relation: Matrix Operations

- **Addition / scalar multiple:** element-wise; requires equal shapes.
- **Transpose:** $(A^T)_{ij} = a_{ji}$. $(AB)^T = B^TA^T$.
- **Product:** $A_{m\times n}B_{n\times p} = C_{m\times p}$ with $c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$ (row i of A dotted with column j of B). Inner dimensions must match. Cost $O(mnp)$.
- **Not commutative:** $AB \neq BA$ in general. But associative and distributive: $(AB)C = A(BC)$, $A(B+C) = AB + AC$.
- **Determinant** (square only): $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$; for 3×3 expand along a row: $\det A = a_{11}M_{11} - a_{12}M_{12} + a_{13}M_{13}$ where M_{1j} is the 2×2 minor. $\det(AB) = \det A \det B$; $\det A^T = \det A$; $|\det A|$ = volume scaling factor of the transformation.
- **Inverse:** A^{-1} exists iff $\det A \neq 0$ (*non-singular*); $AA^{-1} = I$. For 2×2: $A^{-1} = \dfrac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. In general $A^{-1} = \operatorname{adj}(A)/\det A$, or by Gauss–Jordan elimination on $[A \mid I]$. $(AB)^{-1} = B^{-1}A^{-1}$.
- **Rank:** number of linearly independent rows (= columns). $\operatorname{rank} A \leq \min(m,n)$; a square matrix is invertible iff full rank.
- **Trace:** $\operatorname{tr} A = \sum a_{ii}$ = sum of eigenvalues.

Worked Example: Product, Determinant and Inverse

$A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, $B = \begin{pmatrix} 1 & 0 \\ 2 & -1 \end{pmatrix}$.

$AB = \begin{pmatrix} 2\cdot 1 + 1\cdot 2 & 2\cdot 0 + 1\cdot(-1) \\ 1\cdot 1 + 3\cdot 2 & 0 + 3(-1) \end{pmatrix} = \begin{pmatrix} 4 & -1 \\ 7 & -3 \end{pmatrix}$, $\quad BA = \begin{pmatrix} 2 & 1 \\ 3 & -1 \end{pmatrix} \neq AB$.

$\det A = 6 - 1 = 5$, so $A^{-1} = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}$. Check: $AA^{-1} = \frac{1}{5}\begin{pmatrix} 6-1 & -2+2 \\ 3-3 & -1+6 \end{pmatrix} = I$. ✓

3×3: $\det \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{pmatrix} = 1(1\cdot 1 - 3\cdot 0) - 2(0\cdot 1 - 3\cdot 2) + 0 = 1 + 12 = 13$.

Solving $A\mathbf{x} = \mathbf{b}$

- If A is square and invertible: unique solution $\mathbf{x} = A^{-1}\mathbf{b}$ (in practice use **Gaussian elimination**, $O(n^3)$, never form the inverse).
- **Cramer's rule** for small systems: $x_i = \det(A_i)/\det(A)$ where A_i has column i replaced by $\mathbf{b}$.
- Consistency: a solution exists iff $\operatorname{rank}(A) = \operatorname{rank}[A \mid \mathbf{b}]$; it is unique iff that rank equals the number of unknowns.
- Overdetermined (more equations than unknowns, e.g. regression): least squares $\mathbf{x} = (A^TA)^{-1}A^T\mathbf{b}$ (Unit III).

Example: $2x + y = 5,\ x + 3y = 10$: $\mathbf{x} = A^{-1}\mathbf{b} = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 5 \\ 10 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 5 \\ 15 \end{pmatrix} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$.

1.2.1 Eigenvalues and Eigenvectors

Definition 1.3. For a square matrix A, a non-zero vector $\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ if $A\mathbf{v} = \lambda\mathbf{v}$: the matrix only *stretches* $\mathbf{v}$, it does not rotate it.

Formula & Mathematical Relation: Finding Eigenvalues

$A\mathbf{v} = \lambda\mathbf{v} \iff (A - \lambda I)\mathbf{v} = \mathbf{0}$, which has a non-zero solution iff

$$\det(A - \lambda I) = 0 \qquad \text{(characteristic equation)}. \tag{7}$$

For 2×2: $\lambda^2 - (\operatorname{tr} A)\lambda + \det A = 0$. Facts: $\sum \lambda_i = \operatorname{tr} A$, $\prod \lambda_i = \det A$. A **symmetric** matrix has real eigenvalues and orthogonal eigenvectors—this is why covariance matrices (which are symmetric) give PCA an orthonormal basis. If A has n independent eigenvectors, $A = PDP^{-1}$ (*diagonalisation*) with D diagonal, and $A^k = PD^kP^{-1}$.

Worked Example: Eigen-decomposition of a Symmetric 2×2

$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Characteristic equation: $\lambda^2 - 4\lambda + 3 = 0 \Rightarrow (\lambda - 3)(\lambda - 1) = 0$, so $\lambda_1 = 3$, $\lambda_2 = 1$.

$\lambda = 3$: $(A - 3I)\mathbf{v} = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix}\mathbf{v} = 0 \Rightarrow v_1 = v_2$, eigenvector $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)$.

$\lambda = 1$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\mathbf{v} = 0 \Rightarrow v_1 = -v_2$, $\mathbf{v}_2 = \frac{1}{\sqrt{2}}(1, -1)$.

Check: $\mathbf{v}_1 \cdot \mathbf{v}_2 = 0$ (orthogonal, as promised for a symmetric matrix); $\operatorname{tr} = 4 = 3{+}1$; $\det = 3 = 3 \cdot 1$. ✓

Where Computer Science Uses This: Matrices in Computing

Images are matrices of intensities (grayscale) or $H \times W \times 3$ tensors (RGB). A convolution filter is a small matrix slid over the image. A neural-network layer is $\mathbf{y} = \sigma(W\mathbf{x} + \mathbf{b})$. Google's PageRank is the dominant eigenvector of the web's link matrix. GPUs exist because matrix multiplication parallelises perfectly.

1.3 Principal Component Analysis (PCA)

The Problem PCA Solves

Data with p features lives in $\mathbb{R}^p$, but most of its *variance* usually lies along a few directions. PCA finds an orthonormal set of directions (**principal components**) ordered by how much variance they capture, and projects the data onto the first k of them. Uses: dimensionality reduction, compression, noise removal, visualisation (Unit IV of CSE252/INT396 uses it as an algorithm; here we do the linear algebra).

Formula & Mathematical Relation: PCA Algorithm

Given data matrix X (n samples $\times$ p features):

1. **Centre** each column: $\tilde{X} = X - \bar{\mathbf{x}}$ (subtract the feature means). Standardise too if features have different units.
2. **Covariance matrix** ($p \times p$, symmetric): $S = \dfrac{1}{n-1}\tilde{X}^T\tilde{X}$, with $S_{jk} = \mathrm{cov}(x_j, x_k)$.
3. **Eigen-decompose** S: eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$ with orthonormal eigenvectors $\mathbf{w}_1, \ldots, \mathbf{w}_p$.
4. **Variance explained** by component i: $\lambda_i / \sum_j \lambda_j$. Choose k so the cumulative ratio exceeds a target (e.g. 95%), or use a scree plot.
5. **Project:** $Z = \tilde{X}W_k$ where $W_k = [\mathbf{w}_1 \cdots \mathbf{w}_k]$ ($p \times k$). Each row of Z is the sample's k new coordinates (*scores*).
6. **Reconstruct** (lossy): $\hat{X} = ZW_k^T + \bar{\mathbf{x}}$.

The first principal component $\mathbf{w}_1$ maximises $\mathrm{Var}(\tilde{X}\mathbf{w})$ subject to $\|\mathbf{w}\| = 1$; the maximum value is λ_1.

Worked Example: PCA by Hand on Six 2-D Points

Data: $(6,6), (4,4), (7,5), (3,5), (5,7), (5,3)$.
Step 1 — centre. Mean $= (30/6, 30/6) = (5,5)$. Centred rows: $(1,1), (-1,-1), (2,0), (-2,0), (0,2), (0,-2)$.
Step 2 — covariance. $\sum \tilde{x}^2 = 1+1+4+4+0+0 = 10$, $\sum \tilde{y}^2 = 1+1+0+0+4+4 = 10$, $\sum \tilde{x}\tilde{y} = 1+1+0+0+0+0 = 2$. With $n-1 = 5$:

$$S = \frac{1}{5}\begin{pmatrix} 10 & 2 \\ 2 & 10 \end{pmatrix} = \begin{pmatrix} 2 & 0.4 \\ 0.4 & 2 \end{pmatrix}.$$

Step 3 — eigen-decompose. $\lambda^2 - 4\lambda + (4 - 0.16) = 0 \Rightarrow \lambda = 2 \pm 0.4$, so $\lambda_1 = 2.4$, $\lambda_2 = 1.6$. Eigenvectors (same pattern as the earlier example): $\mathbf{w}_1 = \frac{1}{\sqrt{2}}(1,1)$, $\mathbf{w}_2 = \frac{1}{\sqrt{2}}(1,-1)$.
Step 4 — variance explained. PC1: $2.4/4.0 = 60\%$; PC2: 40%.
Step 5 — project onto PC1. $z = \tilde{\mathbf{x}} \cdot \mathbf{w}_1$: $(1,1) \to \sqrt{2} \approx 1.41$; $(-1,-1) \to -1.41$; $(2,0) \to 1.41$; $(-2,0) \to -1.41$; $(0,2) \to 1.41$; $(0,-2) \to -1.41$.
Step 6 — reconstruct $(7,5)$ from its PC1 score alone: $1.41 \cdot \frac{1}{\sqrt{2}}(1,1) + (5,5) = (1,1) + (5,5) = (6,6)$. Error $= (1,-1)$, which lies entirely along PC2—the discarded direction. Sanity check: variance of the scores $= \frac{6 \cdot 2}{5} = 2.4 = \lambda_1$. ✓

Common Pitfall & Misconception: PCA Traps

- **Forgetting to centre:** the "first component" then points at the mean, not along the variance.

- **Not scaling features with different units:** a feature measured in metres will dominate one measured in kilometres purely by magnitude.
- **PCA is unsupervised:** it never looks at class labels. High-variance directions are not necessarily discriminative—that is LDA's job.
- Eigenvector *signs* are arbitrary: $\mathbf{w}$ and $-\mathbf{w}$ are equally valid.

1.4 Linear Discriminant Analysis (LDA)

Fisher's Criterion

LDA is *supervised*: given samples with class labels, find the direction $\mathbf{w}$ onto which projection best **separates the classes**. Fisher's criterion maximises

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} = \frac{(\text{distance between projected class means})^2}{(\text{sum of projected within-class scatters})},$$

i.e. push the class means apart *and* make each class compact.

Formula & Mathematical Relation: LDA for Two Classes

With class means $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2$:

$$S_i = \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T \qquad \text{(scatter of class } i\text{)} \tag{8}$$

$$S_W = S_1 + S_2 \qquad \text{(within-class scatter)} \tag{9}$$

$$S_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T \qquad \text{(between-class scatter)} \tag{10}$$

$$\mathbf{w}^* \propto S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \qquad \text{(optimal direction)} \tag{11}$$

Classify a new $\mathbf{x}$ by its projection $\mathbf{w}^T\mathbf{x}$ against the midpoint threshold $\mathbf{w}^T(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)/2$. For C classes, LDA gives at most $C - 1$ directions: the eigenvectors of $S_W^{-1} S_B$.

Worked Example: Two-Class LDA in 2-D

Class 1: $(1,3), (2,2), (3,4), (2,4), (2,2)$. Class 2: $(5,5), (6,7), (7,6), (6,5), (6,7)$.
Means: $\boldsymbol{\mu}_1 = (10/5, 15/5) = (2,3)$; $\boldsymbol{\mu}_2 = (30/5, 30/5) = (6,6)$.
Scatter of class 1. Deviations: $(-1,0), (0,-1), (1,1), (0,1), (0,-1)$. $\sum d_x^2 = 2$, $\sum d_y^2 = 4$, $\sum d_x d_y = 1$. So $S_1 = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$.
Scatter of class 2. Deviations: $(-1,-1), (0,1), (1,0), (0,-1), (0,1)$. $\sum d_x^2 = 2$, $\sum d_y^2 = 4$, $\sum d_x d_y = 1$. So $S_2 = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$.
Within-class scatter: $S_W = \begin{pmatrix} 4 & 2 \\ 2 & 8 \end{pmatrix}$, $\det S_W = 32 - 4 = 28$, $S_W^{-1} = \frac{1}{28}\begin{pmatrix} 8 & -2 \\ -2 & 4 \end{pmatrix}$.
Direction: $\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2 = (-4, -3)$.

$$\mathbf{w} \propto \frac{1}{28}\begin{pmatrix} 8 & -2 \\ -2 & 4 \end{pmatrix}\begin{pmatrix} -4 \\ -3 \end{pmatrix} = \frac{1}{28}\begin{pmatrix} -32+6 \\ 8-12 \end{pmatrix} = \frac{1}{28}\begin{pmatrix} -26 \\ -4 \end{pmatrix} \propto \begin{pmatrix} 13 \\ 2 \end{pmatrix}.$$

Interpretation. The between-class direction alone is $(4,3)$, but the classes spread more in y (scatter 8 vs 4), so LDA tilts the axis toward x, where the classes are tighter: $\mathbf{w} = (13,2)$.
Classify. Projected means: $\mathbf{w}^T\boldsymbol{\mu}_1 = 26 + 6 = 32$; $\mathbf{w}^T\boldsymbol{\mu}_2 = 78 + 12 = 90$; threshold $= 61$. New

point $(4,5)$: $52+10=62>61 \Rightarrow$ class 2 (just barely). New point $(4,4)$: $52+8=60 \Rightarrow$ class 1.

Aspect	PCA	LDA
Supervision	Unsupervised (ignores labels)	Supervised (needs labels)
Objective	Maximise variance of projection	Maximise class separation / within-class spread
Matrix decomposed	Covariance S	$S_W^{-1}S_B$
Max components	$\min(n-1,p)$	$C-1$ (number of classes minus one)
Typical use	Compression, visualisation, denoising	Feature extraction before classification, face recognition (Fisherfaces)

Table 1: PCA versus LDA.

1.5 Image Transformation

Images as Matrices

A grayscale image is a matrix $I \in \{0,\dots,255\}^{H\times W}$; $I(r,c)$ is the intensity at row r, column c. Colour images stack three such matrices (R, G, B). Row index increases *downward*. A **geometric transformation** moves pixels to new positions: $(x,y)\mapsto(x',y')$. Point operations (brightness, contrast) instead change intensities in place: $I'=\alpha I+\beta$, clipped to $[0,255]$.

Formula & Mathematical Relation: Homogeneous Coordinates and Affine Transformations

Represent a point as $(x,y,1)^T$ so that *translation becomes a matrix multiplication* and all transformations compose by multiplying 3×3 matrices:

$$\text{Translate by } (t_x,t_y): \begin{pmatrix}1&0&t_x\\0&1&t_y\\0&0&1\end{pmatrix} \qquad \text{Scale by } (s_x,s_y): \begin{pmatrix}s_x&0&0\\0&s_y&0\\0&0&1\end{pmatrix}$$

$$\text{Rotate by } \theta \text{ (anticlockwise)}: \begin{pmatrix}\cos\theta&-\sin\theta&0\\\sin\theta&\cos\theta&0\\0&0&1\end{pmatrix} \qquad \text{Shear } (x \text{ by } k): \begin{pmatrix}1&k&0\\0&1&0\\0&0&1\end{pmatrix}$$

$$\text{Reflect in x-axis}: \begin{pmatrix}1&0&0\\0&-1&0\\0&0&1\end{pmatrix} \qquad \text{General affine}: \begin{pmatrix}a&b&t_x\\c&d&t_y\\0&0&1\end{pmatrix}$$

An affine map preserves straight lines and parallelism. Composition: to apply T_1 then T_2, use $M=T_2T_1$ (**right-most acts first**). Inverse transformation: M^{-1}. A rotation matrix R is orthogonal, so $R^{-1}=R^T=R(-\theta)$ and $\det R=1$ (no size change).

Worked Example: Rotate About the Image Centre

Rotate the point $P=(3,1)$ by 90° about the centre $C=(2,2)$. Sequence: translate C to origin, rotate, translate back: $M=T(2,2)\,R(90^\circ)\,T(-2,-2)$.

$$R(90^\circ)=\begin{pmatrix}0&-1&0\\1&0&0\\0&0&1\end{pmatrix} \text{ (since } \cos 90^\circ=0,\ \sin 90^\circ=1).$$

Step by step on P: $T(-2,-2)$: $(3,1) \to (1,-1)$. Rotate: $(1,-1) \to (0{\cdot}1{-}1{\cdot}(-1),\ 1{\cdot}1{+}0) = (1,1)$. $T(2,2)$: $(1,1) \to (3,3)$. So $P' = (3,3)$.

As one matrix: $M = \begin{pmatrix} 0 & -1 & 4 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (check: $M(3,1,1)^T = (-1+4, 3, 1) = (3,3,1)$ ✓).

Worked Example: Scaling Then Translating vs. Translating Then Scaling

S = scale by 2, T = translate by $(1,0)$, point $(1,1)$. TS: scale first $\to (2,2)$, then translate $\to (3,2)$. ST: translate first $\to (2,1)$, then scale $\to (4,2)$. Different results: matrix multiplication is not commutative, and the order of operations matters in graphics pipelines.

Forward vs. Inverse Mapping (why interpolation is needed)

- **Forward mapping** sends each source pixel to $M(x,y)$. Results land on non-integer coordinates and some destination pixels receive no value (*holes*), others several.
- **Inverse mapping** (what every library does): for each destination pixel (x',y') compute the source location $(x,y) = M^{-1}(x',y')$ and *interpolate* the source image there. No holes, but the source location is fractional—hence interpolation.

1.6 Image Interpolation

Definition 1.4. Interpolation estimates the intensity at a non-integer location (x,y) from the known intensities at surrounding integer pixel positions. It is required whenever an image is resized, rotated, warped or registered.

The Three Standard Methods

- **Nearest neighbour:** take the value of the closest pixel, $I(\text{round}(x), \text{round}(y))$. Fastest; produces blocky, jagged results; preserves exact original values (good for label maps / segmentation masks).
- **Bilinear:** weighted average of the 2×2 surrounding pixels, weights proportional to proximity (linear in x, then in y). Smooth; slight blurring; the default for real-time graphics.
- **Bicubic:** fits cubic polynomials through the 4×4 neighbourhood (16 pixels). Sharper than bilinear with fewer artefacts; costlier. Default in Photoshop and for high-quality resampling.

Formula & Mathematical Relation: Bilinear Interpolation

Let (x,y) lie in the cell with corners $(x_0,y_0), (x_1,y_0), (x_0,y_1), (x_1,y_1)$ where $x_1 = x_0 + 1$, $y_1 = y_0 + 1$. Put $a = x - x_0$, $b = y - y_0$ (both in $[0,1]$). Then

$$I(x,y) \approx (1-a)(1-b)\, I(x_0,y_0) + a(1-b)\, I(x_1,y_0) + (1-a)b\, I(x_0,y_1) + ab\, I(x_1,y_1). \quad (12)$$

Each weight is the area of the rectangle *opposite* the corner; the four weights sum to 1.

Worked Example: Bilinear Interpolation at a Fractional Location

Pixel values: $I(0,0) = 10$, $I(1,0) = 20$, $I(0,1) = 30$, $I(1,1) = 40$. Estimate $I(0.25, 0.75)$.
$a = 0.25$, $b = 0.75$. Weights: $(1-a)(1-b) = 0.75 \cdot 0.25 = 0.1875$; $a(1-b) = 0.0625$; $(1-a)b = 0.5625$; $ab = 0.1875$. Sum $= 1$. ✓

$$I = 0.1875(10) + 0.0625(20) + 0.5625(30) + 0.1875(40) = 1.875 + 1.25 + 16.875 + 7.5 = 27.5.$$

Nearest neighbour would pick $(0, 1)$ (the closest corner) and return 30. The bilinear answer 27.5 is nearer to the top-left corner value (30) than to any other because $(0.25, 0.75)$ is nearest that corner—but it is smoothed by the others.
Same thing in two 1-D steps: interpolate along x on row $y = 0$: $10 + 0.25(20 - 10) = 12.5$; on row $y = 1$: $30 + 0.25(10) = 32.5$; then along y: $12.5 + 0.75(32.5 - 12.5) = 27.5$. ✓

Worked Example: Upscaling a 1-D Signal 2×

Row of pixels $[10, 20, 40]$ resized to 5 samples at positions $0, 0.5, 1, 1.5, 2$. Nearest neighbour (round half up): $[10, 20, 20, 40, 40]$. Linear: $[10, 15, 20, 30, 40]$. Linear interpolation produces the intermediate values 15 and 30; nearest neighbour duplicates pixels, giving the "blocky" look.

Common Pitfall & Misconception: Interpolation Cannot Add Information

Enlarging a 100×100 image to 1000×1000 with bicubic interpolation gives a smooth image, not a *detailed* one—no method recovers detail that was never captured. Also: repeated rotations by small angles with interpolation each time accumulate blur; compose the transformations into one matrix and resample once.

Where Computer Science Uses This: Where These Operations Live

`cv2.warpAffine(img, M[:2], (w, h), flags=cv2.INTER_LINEAR)` applies a 2×3 affine matrix with bilinear interpolation. `cv2.resize` takes `INTER_NEAREST`, `INTER_LINEAR`, `INTER_CUBIC`. Data augmentation for CNNs is random affine transforms. GPUs implement bilinear texture sampling in hardware. Medical image registration solves for the M that best aligns two scans. Homographies (3×3 projective, last row not $(0, 0, 1)$) handle perspective for panorama stitching.

1.7 Unit II Quick-Revision Sheet

Exam Tip & Key takeaway: Ten Things to Carry Into the Exam

1. $\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$; zero $\iff$ orthogonal; projection $= \frac{\mathbf{v}\cdot\mathbf{u}}{\|\mathbf{u}\|^2}\mathbf{u}$.
2. $(AB)_{ij}$ = row i of A · column j of B; $AB \neq BA$; $(AB)^T = B^T A^T$; $(AB)^{-1} = B^{-1}A^{-1}$.
3. 2×2 inverse: swap diagonal, negate off-diagonal, divide by $ad - bc$. Singular $\iff$ det $= 0 \iff$ rank deficient.
4. Eigenvalues from $\det(A - \lambda I) = 0$; for 2×2: $\lambda^2 - \operatorname{tr}\lambda + \det = 0$. Symmetric $\Rightarrow$ real λ, orthogonal eigenvectors.
5. PCA: centre $\to$ covariance $S = \tilde{X}^T\tilde{X}/(n-1) \to$ eigenvectors $\to$ project $Z = \tilde{X}W_k$. Variance explained $= \lambda_i / \sum\lambda$.
6. LDA: $\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$; supervised; at most $C - 1$ axes.
7. Homogeneous coordinates make translation a matrix; compose right-to-left; rotate about a point = translate, rotate, translate back.
8. $R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$, $R^{-1} = R^T$, $\det R = 1$.
9. Libraries use *inverse* mapping + interpolation to avoid holes.
10. Bilinear weights $= (1-a)(1-b),\ a(1-b),\ (1-a)b,\ ab$; they sum to 1. Nearest = blocky; bicubic = 16 pixels, sharpest.